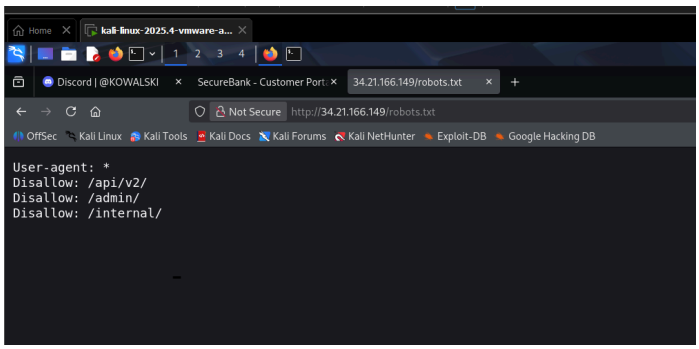
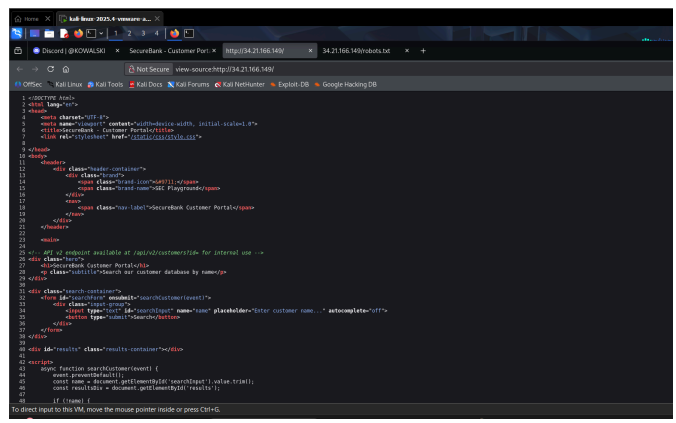
Report 2: SQL Injection

Executive Summary A SQL Injection (SQLi) vulnerability was identified in the Customer Portal of the SecureBank application. The vulnerability exists in the customer search functionality and allows an attacker to manipulate backend database queries. I successfully exploited this to bypass search filters and extract sensitive data from hidden tables, including administrative "secrets."

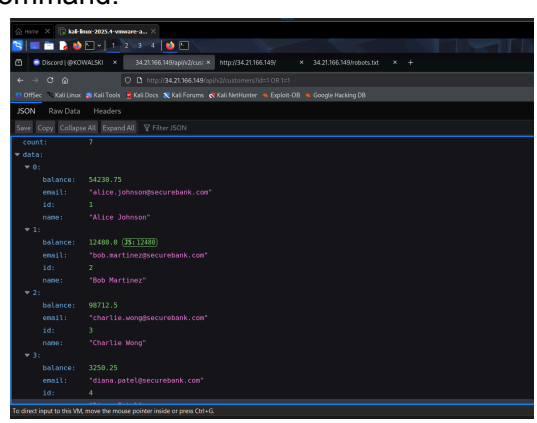
Technical Findings

- **Vulnerability Identification** During initial reconnaissance, I checked the `robots.txt` file which revealed restricted API endpoints at `/api/v2/`. By inspecting the page source code, I identified a search function that makes requests to this backend API.

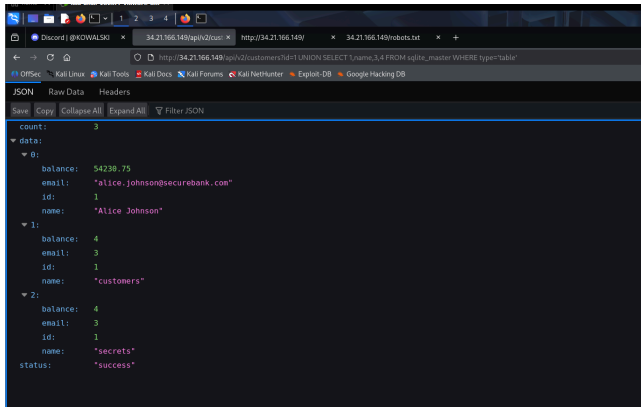




- **Exploitation Steps**
  1. **Testing for SQLi:** I tested the search input using a classic "always true" statement: *1 OR 1=1*. The application responded by returning every single customer record in the database, which confirmed that the input was being executed as a command.



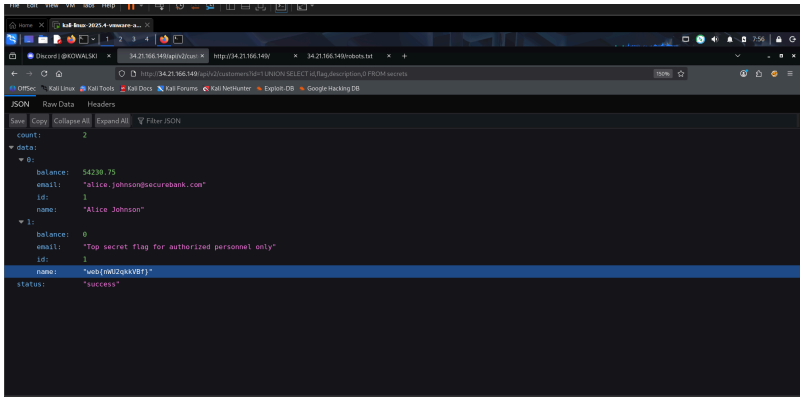
2. **Database Enumeration:** To see what else was in the database, I used a UNION-based payload to identify table names from the `sqlite_master` table.
    - Payload: *`$Id=1$ UNION SELECT 1, name, 3, 4 FROM sqlite_master WHERE type='table'`*



-

3. **Identifying Targets:** This enumeration revealed a hidden table named `secrets`.

4. **Data Extraction:** I crafted a final payload to dump the contents of that `secrets` table. This successfully pulled the hidden flag from the database.
    - Payload: *`$Id=1$ UNION SELECT id, flag, description, 0 FROM secrets`*



Impact

- **Severity: Critical**
- **Reasoning:** This vulnerability is critical because it gives an attacker full access to the database. They can read private customer information, steal system secrets, or even modify/delete data. It also allows an attacker to bypass standard login screens or escalate their privileges within the system.

Remediation

- **Parameterized Queries (Prepared Statements):** This is the most important fix. The developers should use prepared statements so that user input is always treated as text and never as part of the SQL command.
- **Use of Safe APIs:** Use modern frameworks (ORMs) that handle database communication safely by default.
- **Principle of Least Privilege:** The database user used by the web app should only have permission to access the tables it absolutely needs, and it should be blocked from seeing system tables like `sqlite_master`.